

Dashboard Project — Technical Documentation

Oct 6, 2026 · @Yuvarani S Kannan

1. Document control

This document describes the internal design of DB Health Checker (DB_Health_Portal), build 1.9.24, for developers, DBAs and support engineers who maintain, deploy or extend it.

Field	Value
Product	DB Health Checker (packaged as DB_Health_Portal.exe)
Build covered	1.9.24 (dist/dashboard_1.9.24_exa.zip)
Document type	Technical design and maintenance reference
Audience	Developers, Oracle DBAs, deployment and support engineers
Companion document	Dashboard Project — User Guide
Source repository	yuvarani_folder (branch main)

Revision history

Version	Date	Author	Change
1.0	2026-10-06	Yuvarani	Initial technical documentation

Conventions

- File references are relative to the project root, e.g. db_connection.py.
- Secret values (passwords, tokens, SMTP credentials) are never reproduced in this document; only key names are listed.

2. Overview

DB Health Checker is a single-user, locally hosted Streamlit application that monitors Oracle production, reporting and standby databases plus their host servers, and sends email alerts.

Purpose

Give DBAs one screen showing whether every database in an estate is up, backed up, in sync and not running out of space, without installing agents on the database servers.

Scope

In scope	Out of scope
Oracle databases reachable over SQL*Net (Thick mode)	Non-Oracle databases
Production, reporting and standby per registry line	Adding or editing databases from the UI (registry file only)
Host metrics over SSH (CPU, memory, mounts, processes, ASM)	Agent-based or SNMP monitoring
Email alerts via SMTP (Microsoft Graph in source only)	Multi-user access, login, role-based access
Windows packaged exe	Server deployment, high availability

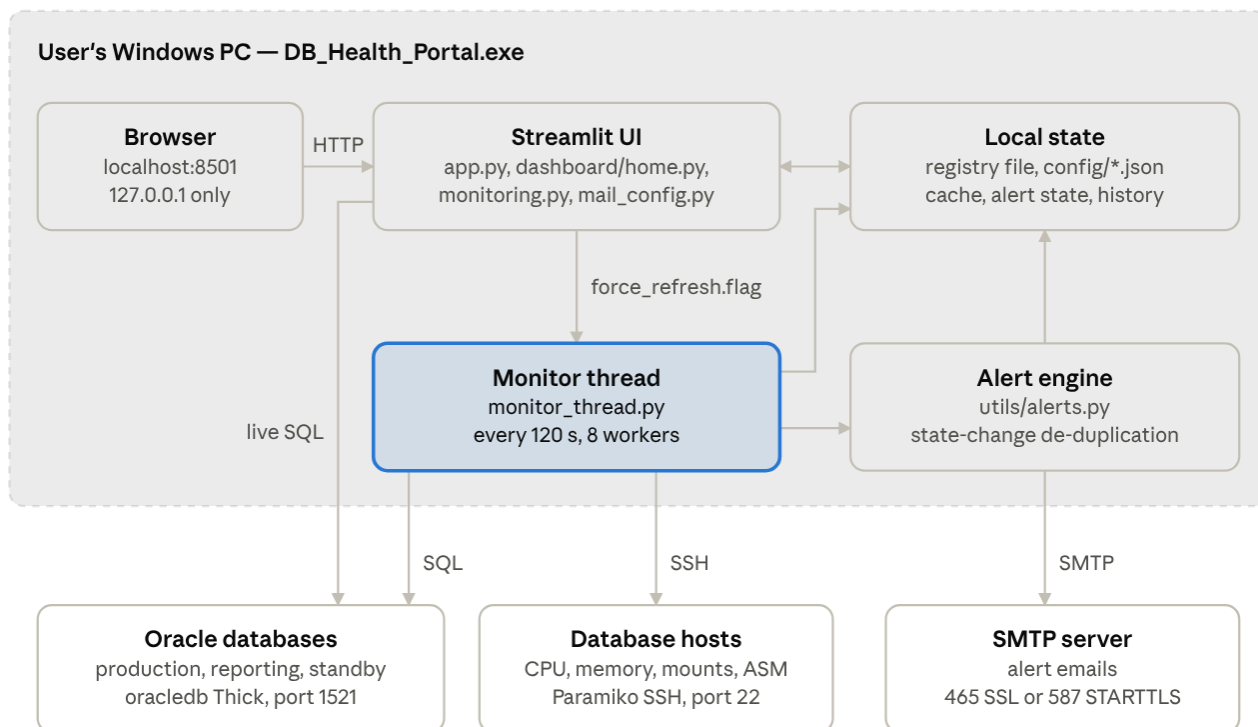
Key capabilities

- Fleet view with Healthy / Warning / Critical classification per database.
- Per-database drill-down: sessions, memory, tablespaces, backups, blocking, alert log, growth, top SQL, FRA, standby sync.
- Operational actions: kill session, RMAN archive-log clean-up.
- Background monitor every 120 seconds, up to 8 databases in parallel, feeding the alert engine.
- Email alerts for 7 evaluated conditions (32 checkboxes shown in the UI; see section 16), configured per registry file, with send history.
- Self-contained Windows distribution with a bundled Oracle Instant Client 19.

3. System architecture

Everything runs inside one Windows process on the user's PC: a Streamlit UI and a background monitor thread share local JSON state and reach out to Oracle, the database hosts and the mail server.

One local process serves the UI and polls every database every 2 minutes



System architecture · 6 local components, 3 external systems

The UI queries databases live for the screens the user is viewing; the monitor thread polls all databases every 120 seconds and feeds the alert engine. The UI writes `force_refresh.flag` to wake the monitor early.

3.1 Request flow

1. The browser loads a page; app.py routes to home.py or monitoring.py.
2. The page opens Oracle connections (Thick mode where required) and SSH sessions in thread pools and renders the results with Plotly.
3. Results are cached in `st.session_state` for up to 60 s per database; fragments re-run every 30 s.

3.2 Monitoring flow

1. `monitor_thread.py` wakes every 120 s, or within 5 s of `force_refresh.flag`.

2. `collect_db_metrics` runs per database, 8 at a time, and writes `config/db_monitor_cache.json`.
3. `utils/alerts.py` compares results with `config/alert_state.json` and emails only on state changes.
4. `generate_db_status_log.py` rewrites `db_status_report.log` at the end of each cycle.

3.3 Concurrency

Locks (`_cache_write_lock`, `_alert_lock`, Thick-mode init lock) are per process. The legacy `monitor_daemon.py` has no locks; running it alongside the app would race on the same files.

4. Technology stack and dependencies

The application is pure Python on Streamlit, talks to Oracle through `oracledb` in Thick mode, and reaches DB hosts over SSH with `Paramiko`; all versions are pinned in `requirements.txt`.

Layer	Component	Pinned version	Used in
Language	Python 3 (3.10 and 3.14 bytecode present)	—	All modules
Web UI	streamlit	1.44.1	<code>app.py</code> , <code>dashboard/</code>
Charts	plotly	6.7.0	<code>dashboard/home.py</code> , <code>dashboard/monitoring.py</code>
Data frames	pandas / numpy	2.3.3 / 2.2.6	<code>db_connection.py</code> , <code>queries/queries.py</code>
Oracle driver	oracledb (Thick mode)	3.4.1	<code>db_connection.py</code> , <code>monitor_daemon.py</code> , <code>generate_db_status_log.py</code>
Oracle client	Oracle Instant Client 19 DLLs	19.x (bundled)	<code>oracle_client/</code>
SSH	paramiko (+ bcrypt, cryptography, PyNaCl)	5.0.0	<code>utils/ssh_*_provider.py</code> , <code>utils/asm_provider.py</code>

Layer	Component	Pinned version	Used in
Local OS metrics	psutil	7.2.2	utils/
Email (SMTP)	smtplib, email.message (stdlib)	—	utils/alerts.py
Email (Microsoft 365)	msgraph-core, azure-identity, msal	1.5.1 / 1.25.3 / 1.37.0	utils/alerts.py (optional, guarded import)
Configuration	python-dotenv	1.2.2	db_connection.py, utils/alerts.py
Packaging	PyInstaller	not pinned	dash_manually.spec
Large files	Git LFS	—	oracle_client/, build output

Dependency notes

- The Microsoft Graph stack (azure, msal, msgraph, kiota) is in requirements.txt but excluded from the PyInstaller build in dash_manually.spec. The packaged exe therefore sends mail over SMTP only.
- matplotlib, altair, pyarrow and sqlalchemy are also excluded from the build; the UI renders charts only through st.plotly_chart.
- The MSVC runtime (msvcp140.dll, vcruntime140.dll, vcruntime140_1.dll) is copied into oracle_client/ at build time because the Instant Client needs it.

5. Project structure

About 16,000 lines of Python and SQL, split into a launcher, a Streamlit UI package, a database layer, a background monitor and SSH/storage providers.

Path	Lines	Responsibility
run_dashboard.py	439	Launcher and PyInstaller entry point; starts Streamlit and the browser
app.py	1,388	Streamlit main script: page routing, registry-file setup, sidebar
db_connection.py	2,091	Oracle connections, registry-file parsing (NC convention), per-DB config lookup

Path	Lines	Responsibility
monitor_daemon.py	484	Background monitoring cycle; writes cache and status files
monitor_thread.py	680	In-process monitoring thread started by the UI
generate_db_status_log.py	276	Standalone DB status log generator
test_oci.py	24	Smoke test for Oracle Instant Client loading
dashboard/home.py	2,790	Home page: per-database production, reporting and standby cards
dashboard/monitoring.py	1,919	Monitoring page: OS resources, mounts, processes, ASM, charts
dashboard/mail_config.py	393	Email/alert configuration page
dashboard/app.py	381	Shared UI helpers
dashboard/styles.css, assets/style.css	—	Custom CSS injected into Streamlit
queries/queries.py	800	SQL text used by the UI and monitor
queries/dashboard_pkg.sql	329	PL/SQL package installed on monitored databases
utils/alerts.py	618	Alert evaluation, de-duplication, SMTP / Microsoft 365 sending
utils/asm_provider.py	539	ASM disk-group metrics
utils/ssh_mount_provider.py	422	Remote filesystem/mount usage over SSH
utils/ssh_process_provider.py	831	Remote top processes over SSH
utils/ssh_resource_provider.py	153	Remote CPU and memory over SSH
utils/storage_provider.py, utils/storage.py	891	Local and remote storage abstraction
config/	—	Runtime JSON state (see section 6)
oracle_client/	—	Bundled Oracle Instant Client 19 DLLs (Git LFS)
.streamlit/config.toml	8	Headless server, no usage stats, minimal toolbar

Path	Lines	Responsibility
dash_manually.spec	104	PyInstaller build definition
build/, dist/	—	Build output; dist/ holds the release zip

Key reference files in the root: README.md (overview), BUILD_EXE_GUIDE.md (packaging) and NC_CONFIG_GUIDE.txt (registry-file format).

6. Configuration

Database connections come only from a 20-column registry file; email settings are stored per registry file in config/; nothing is configured through environment variables at runtime.

6.1 Registry file

Accepted formats: .txt (whitespace-separated), .csv, .xlsx, .xls. One line per database. Parsed by `parse_smart_txt_to_df()` in `db_connection.py` and cached in memory by file modification time.

Group	Columns	Fields
Production	1-8	db_name, host, port, service_name, username, password, host_username, host_password
Reporting	9-14	reporting_db_name, reporting_host, reporting_port, reporting_service_name, reporting_username, reporting_password
Standby	15-20	standby_db_name, standby_host, standby_port, standby_service_name, standby_username, standby_password

NC rule: a single NC token in place of a group's first field skips the whole group and consumes one token only. NC for production removes the line from the Home page. `get_reporting_db_config()` returns None and `get_standby_db_config()` skips when their name is nc.

Username mapping (`get_oracle_mode`): sys connects as SYSDBA; sysoper, sysbackup, sysdg and syskm connect with their matching privilege.

6.2 Registry path

db_path_config.json holds txt_path (active file), history (last 10 files) and pending_txt_path. Uploaded files are stored in scratch/. Changing the registry writes config/force_refresh.flag.

6.3 Email configuration

File: config/<registry-file-basename>.json, or config/email_config.json when no registry is loaded (utils/alerts.get_custom_config_path).

Key	Type	Notes
smtp_server, smtp_port	string, int	465 uses implicit TLS (SMTP_SSL)
smtp_username, smtp_password	string	Stored in plaintext
from_email	string	Also the Mail History username
to_emails	string	Comma-separated
enabled	bool	Alerts disabled when no config exists
preferences	object	32 boolean alert-type keys (db_down, listener_down, tablespace_critical, archive_log_full, blocking_sessions, ...), default all false

6.4 Other settings

Setting	Location	Value
Streamlit server	.streamlit/config.toml	headless = true, gatherUsageStats = false, toolbarMode = minimal
Bind address and port	run_dashboard.py	127.0.0.1, 8501 then 8502–8519
Monitor poll interval / parallelism	monitor_thread.py	POLL_INTERVAL = 120 s, 8 workers
TNS_ADMIN	environment	Optional; tnsnames.ora passed as config_dir in Thick mode

Setting	Location	Value
SSH keys	registry key path, keys/, ~/.oci, ~/.ssh	keys/ is referenced by the spec but absent from the working tree
.env, config/db_config.json	root, config/	ORACLE_USER, ORACLE_PASS, ORACLE_DSN / user, password, dsn — present but not read by current code

6.5 Open question

Commit 8c5435c and a023632 mention prefix-based password patterns for m5 and fa service names (e.g. fa182.dbprod01.csi.waynepa). No such logic exists in the current yuvarani_folder source. Confirm whether it was removed or lives elsewhere before documenting it.

7. Core modules

The live path is run_dashboard.py → app.py → dashboard/home.py and dashboard/monitoring.py for display, with monitor_thread.py feeding utils/alerts.py in the background; monitor_daemon.py and dashboard/app.py are legacy.

7.1 UI layer

Module	Key behaviour
app.py	Routes on st.session_state.page (home, monitoring, mail_history, mail_config); builds the sidebar; starts monitor_thread; Mail History page and gate; idle auto-exit and shutdown
dashboard/home.py	Setup screen; portal banner; KPI ribbon; database cards; host resources. load_db_status_summary_basic runs inline SQL with up to 30 threads; reporting checks max 5 parallel (subprocess); standby max 5 parallel. @st.fragment(run_every=30) refreshes the card area

Module	Key behaviour
dashboard/monitoring.py	Per-database detail: 18 parallel query fetches, each on its own connection, cached in session state (mon_cache_<db>) for 60 s; kill session; RMAN archive clean-up over SSH; 30 s process fragment
dashboard/mail_config.py	SMTP form, test email, 32 alert preferences in 9 groups
dashboard/app.py	Older standalone variant; not on the launch path

No st.cache_data or st.cache_resource is used; all caching is manual in st.session_state.

7.2 Health classification (home.py)

Category	Condition (any)
Critical	DB DOWN, listener DOWN, backup FAILED, log_gap_alert
Warning	Backup PENDING/WARNING, tablespace $\geq 90\%$, blocking sessions, deadlock, sessions over limit, alert-log error
Healthy	None of the above

7.3 Background monitor (monitor_thread.py)

- Singleton daemon thread started by app.py; cooperative stop via _stop_event.
- Every 120 s, collect_db_metrics runs for each database in a ThreadPoolExecutor(8).
- After each database: rewrite config/db_monitor_cache.json under _cache_write_lock; evaluate alerts under _alert_lock.
- After each cycle: regenerate db_status_report.log via generate_db_status_log.generate_report().
- Waits in 5 s steps and starts early when config/force_refresh.flag exists; deletes the flag at cycle start.
- The Home cards do not read the cache for status; they query live. The cache file's modification time drives the "last cycle" caption.

7.4 Metrics collected

Metric	Source	Alert rule
Database up	Connection success	DOWN on failure
Listener	<code>v\$listener_network</code> ; else TCP probe to port (2 s)	DOWN if no rows / port closed
Backup	<code>v\$rman_backup_job_details</code> , latest job	SUCCESS/FAILED only if ≤ 1 day old, else PENDING; no email for FAILED
Tablespaces	<code>dba_data_files</code> + <code>dba_free_space</code> (max incl. autoextend)	$\geq 90\%$ critical; TEMP/UNDO excluded
Archive / FRA	<code>v\$archive_dest</code> , <code>v\$recovery_file_dest</code> , <code>v\$archived_log</code>	$\geq 90\%$, or ORA-00257 / archiver error
Blocking sessions	<code>v\$session</code> where blocking_session is not null	Any row
Deadlock	<code>v\$diag_alert_ext</code> (last hour); fallback <code>v\$session_wait</code> row-lock waits	Any match
ORA errors	<code>v\$diag_alert_ext</code> , last 48 h, top 50	One email per distinct ORA code
Oracle CPU / memory	<code>v\$osstat</code> , <code>v\$sysmetric</code> , <code>v\$sga</code> , <code>v\$process</code>	Display only
Sessions	<code>v\$parameter</code> sessions vs count	$> \max(50, 85\% \text{ of limit})$, tooltip only
Data Guard	<code>V\$ARCHIVE_DEST_STATUS</code> , <code>V\$ARCHIVED_LOG</code> ; standby gap over SSH	Gap ≥ 2 sequences
Reporting DB	Subprocess check	UP / DOWN
Mount points	df -PkT and /proc/mounts over SSH	$\geq 90\%$ (not collected by the monitor loop)
ASM	<code>v\$asm_diskgroup</code> via sqlplus / as sysasm over SSH	Display only, 60 s cache
Host CPU / RAM	nproc, free, vmstat or top over SSH	Display only

Metric	Source	Alert rule
Top processes	ps two-sample delta; fallback v\$sesstat	Display only, 30 s cache

7.5 Providers (utils/)

Module	Function
alerts.py	Alert state machine, SMTP sending, history and toast queues
asm_provider.py	Logs in as opc, sudo su - grid, runs ASM discovery script and v\$asm_diskgroup
ssh_mount_provider.py	Remote df; alternative path via DBMS_SCHEDULER + UTL_FILE
ssh_process_provider.py	Remote ps sampling; v\$sesstat fallback
ssh_resource_provider.py	Remote CPU and memory
storage_provider.py	Storage abstraction; Java/scheduler-based file access
storage.py	Hard-coded remote mount test; runs on import (see section 9)

7.6 Alert engine (utils/alerts.py)

- State-change de-duplication: each alert sends once and re-arms when the condition clears. State per host:service in config/alert_state.json.
- ORA codes already sent are remembered (last 50) and not re-sent.
- Exception: the Data Guard log-gap email in monitor_thread.py has no de-duplication and repeats every cycle (2 min) while the gap lasts.
- Each attempt is recorded in email_history.json (last 200; timestamp, subject, alert_type, db_name, server, to, status, error, body_preview) and email_notifications.json (last 20, for toasts); full text goes to alerts_log.txt.

8. Data access

All database access uses python-oracledb with Easy Connect strings and read-only queries against `V$` and `DBA_` views, apart from the kill-session and archive clean-up actions.

8.1 Driver mode

- Thin mode is the default. Thick mode is initialised lazily by `_ensure_thick_mode()` in `db_connection.py`, guarded by a lock with a double check.
- `generate_db_status_log.py` enables Thick mode at import; reporting-DB checks run Thick mode in a child process to avoid DPY-2019 (mode already set).
- Instant Client search order: `_internal/oracle_client` → `./oracle_client` → `sys._MEIPASS/oracle_client` → any PATH folder with `oci.dll` → `%ORACLE_HOME%\bin` → three machine-specific paths.

8.2 Connections

Aspect	Implementation
Connect string	host:port/service_name; on DPY-6001, ORA-12505 or ORA-12154 retries once with a SID descriptor
Pool	One global <code>create_pool(min=1, max=5, increment=1)</code> for the database selected in the UI; rebuilt on selection change
Fallback chain	<code>get_db_connection()</code> : pool → direct connect → <code>SSHConnection</code> (sqlplus over SSH)
Monitor / daemon	Direct <code>oracledb.connect</code> per check, no pool
Connect timeouts	2-5 s <code>tcp_connect_timeout</code> depending on caller; reporting subprocess 15 s
Query timeout	None (<code>call_timeout</code> not set)
Retry	SID fallback only; the next 120 s cycle acts as retry

8.3 SQL sources

- `queries/queries.py` — inline SQL used by the UI and monitor (the active source).
- `queries/dashboard_pkg.sql` — PL/SQL package with 26 read-only procedures returning `SYS_REFCURSOR`, grouped as status and sessions, storage, backup, host and memory, listener and alert log. `get_db_growth` reads the custom table

DASHBOARD_GROWTH_TRACK.

- The package is only called from monitor_daemon.py, and that call is broken. It has drifted from queries.py (no type='USER' filter in get_session_stats; duplicate rows in get_cpu_sessions; different aliases in get_pid_sid_map).

8.4 Write operations on monitored systems

Action	Statement	Where
Kill session	ALTER SYSTEM KILL SESSION 'sid,serial#' IMMEDIATE	dashboard/monitoring.py
Archive clean-up	RMAN DELETE NOPROMPT ARCHIVELOG ALL COMPLETED BEFORE 'SYSDATE-15' over SSH	dashboard/monitoring.py
Mount check (alternative path)	CREATE OR REPLACE DIRECTORY on /tmp, DBMS_SCHEDULER jobs running /bin/bash	utils/ssh_mount_provider.py
Storage provider	Java source and dbms_java.grant_permission	utils/storage_provider.py

8.5 Data model in Python

Every query result is a pandas DataFrame. Per-database monitor results are dictionaries keyed by db_name in config/db_monitor_cache.json; alert state is keyed host:service with flags db_down, listener_down, tablespaces, arc_log_alert, mount_points, ora_errors_sent, deadlock_alert and blocking_alert.

9. Security considerations

The application binds to 127.0.0.1 and has no remote attack surface, but credentials are handled in plaintext throughout and two production secrets are hard-coded in tracked source; rotate them before this repository or build is shared further.

9.1 Controls in place

- Streamlit listens on 127.0.0.1 only; usage statistics disabled.
- Standby SSH scripts escape values with shlex.quote.
- SMTP on port 465 uses SMTP_SSL with ssl.create_default_context().
- Mail History requires the sender email and SMTP password (plaintext comparison).

9.2 Findings

#	Severity	Issue	Location	Recommended fix
1	Critical	Hard-coded Office 365 SMTP account and password; saved SMTP settings ignored for any port other than 465	utils/alerts.py:293-297	Remove; use configured settings with STARTTLS and timeout; rotate the password
2	Critical	Hard-coded SSH host, user and password; connects on import	utils/storage.py:4-6, 33	Delete module or move to config; rotate the password
3	Critical	Secrets tracked in git: .env, config/*.json with passwords, dist/ copies; no .gitignore	repository	Add .gitignore, remove from index, purge history, rotate all exposed credentials
4	High	DB and host passwords written to normal_db_debug.log on every DOWN event	db_connection.py:94, 112, 1230-1247	Mask secrets in logs
5	High	reporting_password stored in config/db_monitor_cache.json and scratch/ result files	monitor_thread.py:416; run_dashboard.py:297, 323-324	Exclude secrets from cache and result files
6	High	SQL injection: user-typed SID concatenated into ALTER SYSTEM KILL SESSION	dashboard/monitoring.py:1541	Validate SID and serial# as integers
7	High	Shell injection: ORACLE_SID and user/password unquoted in remote bash	db_connection.py:1879-1883	shlex.quote all values
8	High	Paramiko AutoAddPolicy in every SSH client (no host-key verification)	9 modules	Use RejectPolicy with a managed known_hosts
9	High	Passwords on remote command lines (sqlplus user/pass, RMAN) visible in ps	db_connection.py:1629, 1879; monitoring.py:375-391	Pass credentials on stdin or use OS authentication / wallet
10	Medium	Monitoring connects as SYS/SYSDBA	db_connection.py:333-346	Dedicated read-only monitoring account with SELECT_CATALOG_ROLE
11	Medium	DDL on monitored DBs: directory on /tmp, scheduler jobs running /bin/bash, Java with <<ALL FILES>> permission	ssh_mount_provider.py:327-351; storage_provider.py:261-491	Remove; use the SSH path only
12	Medium	opc password sent to sudo without waiting for a prompt; reused for SSH	asm_provider.py:348-351	Key-based SSH and NOPASSWD sudo for a fixed command
13	Medium	Registry, SMTP and history files stored in plaintext	config/, registry file	Windows DPAPI or OS keyring for secrets
14	Low	SSH agent and default keys used automatically	db_connection.py:1814-1832	Restrict to configured key

10. Build, deployment and runtime

The release is a PyInstaller one-dir bundle built from dash_manually.spec with run_dashboard.py as the entry point; it is zipped and copied to each user's Windows PC.

10.1 Development setup

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python run_dashboard.py
```

Software to install (developer / build machine)

End users of the packaged exe need none of this; it is only for running from source or building a release.

Software	Version	Required for	Official download
Python (64-bit)	3.10.x (build machine runs 3.10.11)	Running from source, building	python.org/downloads/windows
Git for Windows	Latest	Cloning the repository	git-scm.com/downloads/win
Git LFS	Latest	Pulling oracle_client/ DLLs and build output	git-lfs.com
Oracle Instant Client Basic (Windows x64)	19.x	Only if oracle_client/ is missing or must be refreshed	Oracle Instant Client for Windows x64
Microsoft Visual C++ Redistributable (x64)	Latest supported	Required by Instant Client; bundled into the build	vc_redist.x64.exe (info)
PyInstaller	Latest stable	Building the exe	pyinstaller.org
Firefox (optional)	Latest	Auto-opened by the launcher	mozilla.org/firefox

Reference documentation: [python-oracledb installation](#), [oracledb on PyPI](#), [Streamlit installation](#).

Step-by-step setup from source (Command Prompt)

1. Install Python 3.10 64-bit and tick "Add python.exe to PATH". Verify:

```
python --version
pip --version
```

2. Install Git and Git LFS, then clone and pull the large files:

```
git lfs install
git clone <repository-url> dash_manually
cd dash_manually\yuvarani_folder
git lfs pull
```

3. Create and activate a virtual environment:

```
python -m venv venv
venv\Scripts\activate
python -m pip install --upgrade pip
```

PowerShell users activate with `venv\Scripts\Activate.ps1`; if blocked, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.

4. Install all pinned libraries:

```
pip install -r requirements.txt
```

5. Or install the core libraries individually (same pinned versions as requirements.txt):

```
pip install oracledb==3.4.1 streamlit==1.44.1 pandas==2.3.3 numpy==2.2.6
plotly==6.7.0
pip install paramiko==5.0.0 psutil==7.2.2 python-dotenv==1.2.2
requests==2.32.5
pip install openpyxl
```

openpyxl (for .xlsx) and xlrd (for .xls) are not in requirements.txt, are not installed on the build machine, and openpyxl is listed in the excludes of dash_manually.spec.

pandas.read_excel (db_connection.py:704–710) therefore fails for Excel registry files in both source and exe runs. Add openpyxl to requirements.txt and remove it from the spec excludes before advertising Excel support.

6. Check that the Oracle client loads (Thick mode):

```
dir oracle_client\oci.dll
python -c "import oracledb;
oracledb.init_oracle_client(lib_dir=r'oracle_client'); print('Thick mode OK',
oracledb.clientversion())"
```

If oracle_client/ is empty, download Instant Client Basic 19 for Windows x64, unzip it, and copy all its DLLs into oracle_client/.

7. Run the dashboard:

```
python run_dashboard.py
```

8. Build the release:

```
pip install pyinstaller
pyinstaller --clean dash_manually.spec
powershell Compress-Archive -Path dist\DB_Health_Portal_App -DestinationPath
dist\dashboard_<version>_exa.zip
```

Verify the installation

```
pip check
python -c "import streamlit, oracledb, paramiko, pandas, plotly;
print('imports OK')"
```

10.2 Build the executable

```
pip install pyinstaller
pyinstaller --clean dash_manually.spec
```

- Output: dist/DB_Health_Portal_App/ containing DB_Health_Portal.exe (console=True, UPX on).
- Bundled data: app.py, db_connection.py, monitor_thread.py, monitor_daemon.py, dashboard/, queries/, utils/, config/, assets/, keys/.
- Binaries: every oracle_client/.dll *plus the MSVC runtime, all under oracle_client/*. Any *other ora.dll* that PyInstaller picks up outside oracle_client/ is filtered out so a system Oracle home cannot shadow the bundled client.
- Excluded: azure, msal, msgraph, kiota, pyarrow, sqlalchemy, altair, matplotlib, test frameworks.
- Package and release: zip dist/DB_Health_Portal_App as dashboard_<version>_exa.zip.

BUILD_EXE_GUIDE.md describes an older launcher.py flow with manual file copying. dash_manually.spec supersedes it and should be treated as the source of truth.

10.3 Startup sequence

1. run_dashboard.py resolves its base directory (sys._MEIPASS when frozen). An app.py placed next to the exe overrides the bundled copy; config/ lives next to the exe.
2. If 8501 already answers /_stcore/health with HTTP 200, it opens the browser and exits; otherwise it takes the first free port from 8502–8519.

3. It writes `config/force_refresh.flag` so the monitor runs a cycle immediately.
4. It starts Streamlit in-process with `headless=true`, `address=127.0.0.1`, `disconnectedSessionTTL=2`, `developmentMode=false`.
5. After 3 s it opens Firefox from the standard install paths, or the default browser.
6. `app.py` exits the process after 10 s with no connected sessions, or on the Shutdown button / `?shutdown` URL (monitor stopped, 2.5 s grace, `os._exit(0)`).

10.4 Helper mode

`run_dashboard.py --check-reporting-db <db> <registry> <result> <log>` runs one reporting-DB check in Thick mode, writes JSON and exits.

`db_connection.check_reporting_db_status` calls it as a subprocess with a 15 s timeout, which isolates Thick-mode hangs from the UI.

10.5 Runtime files

File	Written by	Purpose
<code>db_path_config.json</code>	<code>db_connection.py</code>	Active registry path, last 10 registries, pending path
<code>config/<registry>.json</code>	<code>mail_config.py</code>	SMTP settings and alert preferences for that registry
<code>config/email_config.json</code>	<code>mail_config.py</code>	Fallback mail config when no registry is loaded
<code>config/db_monitor_cache.json</code>	<code>monitor_thread.py</code>	Last monitor results per database
<code>config/daemon_status.json</code>	<code>monitor_thread.py</code>	fetching, <code>last_cycle_start</code> , <code>last_cycle_end</code>
<code>config/alert_state.json</code>	<code>utils/alerts.py</code>	Per host:service alert state for de-duplication
<code>config/email_history.json</code>	<code>utils/alerts.py</code>	Last 200 send attempts
<code>config/email_notifications.json</code>	<code>utils/alerts.py</code>	In-app toast queue
<code>config/force_refresh.flag</code>	UI, launcher	Wakes the monitor early
<code>alerts_log.txt</code>	<code>utils/alerts.py</code>	Text alert log; Mail History fallback
<code>db_status_report.log</code>	<code>monitor_thread.py</code>	Regenerated status report

File	Written by	Purpose
scratch/	UI, launcher	Uploaded registries, session_history.csv, reporting-check results

11. Logging, error handling and troubleshooting

The code uses `print()` and direct file appends rather than the logging module; nothing rotates, and broad except blocks turn most failures into DOWN, UNKNOWN or empty values.

11.1 Log files

File	Content	Rotation
oracle_thick_mode_debug.log	Thick-mode init, ASM and standby diagnostics	None
normal_db_debug.log	Primary, reporting, standby and SSH status (contains passwords, see 9.2 #4)	None
alerts_log.txt	Full text of every alert email	None
db_status_report.log	Status report, overwritten each cycle	Overwrite
Console window	<code>print()</code> output from all modules	—

11.2 Error conventions

- Oracle errors are normalised to ORA-NNNNN strings.
- Remote scripts emit `##STAGE_ERROR##` markers with exit codes 90–95; known ORA codes map to readable messages.
- `monitor_thread._run_query` returns safe defaults on any exception.

11.3 Troubleshooting

Symptom	Check
DPI-1047 / Thick mode fails	oracle_client/ present with oci.dll and MSVC DLLs; see oracle_thick_mode_debug.log; run test_oci.py (hard-coded lib_dir)

Symptom	Check
DPY-2019	Thick/Thin mode switched after first connect; keep Thick checks in the subprocess path
ORA-12505 / ORA-12154	Service name vs SID; the code retries with SID once
All databases DOWN	Network/firewall to port 1521; registry columns shifted by a misplaced NC
Host metrics empty	SSH port 22, key path, host_username/host_password; AutoAddPolicy hides host-key changes
ASM empty	opc login and sudo to grid on the host
Emails not sent with port 587	Hard-coded SMTP block overrides settings (9.2 #1)
Repeated log-gap emails	Expected with current code: no de-duplication for that alert
Monitor caption stale	config/daemon_status.json timestamps; create config/force_refresh.flag
Port already in use	Another instance on 8501; launcher reuses it or picks 8502-8519

12. Known limitations and future enhancements

Beyond the security fixes in section 9, the main gaps are silent functional bugs hidden by broad exception handling and two parallel monitor implementations.

12.1 Known defects

Area	Defect	Location
Data Guard	log_gap_alert reset to False after the destination-error check set it, so destination errors are lost	monitor_thread.py:400, 431
Mount alerts	Monitor loop never collects drives, so the mount-point alert cannot fire	monitor_thread.py
Tablespace warning	tablespace_warning preference exists but no 80% rule is implemented	utils/alerts.py

Area	Defect	Location
Backup FAILED	Shown on the card but never emailed	monitor_thread.py
Legacy daemon	Wrong FRA column names (arc_pct always 0), missing MESSAGE column, execute_proc_to_df not imported	monitor_daemon.py:173-174, 191, 215
Storage provider	Imports get_db_names, which does not exist; SSH process path never runs	utils/storage_provider.py:514
SSH fallback	monitor_thread requires host_password but _open_ssh_client is key-only	monitor_thread.py:115
Session chart	New session_history.csv seeded with random points	dashboard/monitoring.py:141-157
Home diagnostics	diag_result preset to ok; registry TCP check never runs	dashboard/home.py:2425-2426
Cache writes	Not atomic; daemon_status.json written without a lock	monitor_thread.py
UI text	Mail History refers to a non-existent "Configure Email Alerts" panel	app.py:1051
Status filter	Multiselect cannot hold all three statuses at once	dashboard/home.py
Pool	Global pool rebuilt without a lock	db_connection.py:1430-1463

Also: Excel registry files (.xlsx/.xls) cannot be read today because openpyxl/xlrd are not installed and openpyxl is excluded from the build (requirements.txt, dash_manually.spec). Use .txt or .csv until fixed.

Also, observed on 2026-10-06 while capturing screenshots with the database unreachable: the Database Health page crashed with `TypeError: string indices must be integers` at `dashboard/monitoring.py:1282` (`get_growth_color(growth_rates["daily"], 1)`). `growth_rates` is a string instead of a dict when the fetch fails [Likely: the per-query connection wrapper returns an error string]. The same run rendered the Home page content without styling below the health panels. Guard `growth_rates` with `isinstance(growth_rates, dict)` before use.

12.2 Design limitations

- Single user per PC; no authentication or roles.

- No query timeout, so one hung database can hold a worker for the whole cycle.
- Microsoft Graph code is imported but never used and is excluded from the build.
- Two SQL sources (queries.py and dashboard_pkg.sql) that have drifted apart.
- BUILD_EXE_GUIDE.md describes an outdated build flow.

12.3 Recommended roadmap

1. Rotate exposed credentials, remove hard-coded secrets, add .gitignore and purge history.
2. Fix the SQL and shell injection points and enable host-key verification.
3. Introduce a least-privilege monitoring account.
4. Remove monitor_daemon.py, dashboard/app.py, utils/storage.py and the unused PL/SQL package, or make the package the single SQL source.
5. Replace print() with the logging module with rotation and secret masking.
6. Set call_timeout on connections; write cache files atomically.
7. Fix the defects in 12.1 and add unit tests for registry parsing and alert de-duplication.

13. System requirements and network ports

The client is any 64-bit Windows PC that can reach each database on its listener port and each host on SSH; monitored databases need Oracle 11.2 or later (12c for storage_provider features).

13.1 Client PC

Item	Requirement
OS	Windows 10/11 64-bit (build is Windows-only; source is cross-platform)
Runtime	None for the exe; Python 3.10+ for source runs
Disk	Space for the unzipped bundle plus logs; logs do not rotate
Browser	Firefox preferred (auto-opened), any current browser works
Local port	One free port in 8501–8519 on 127.0.0.1
Write access	The exe folder (config/, scratch/, logs)

13.2 Monitored systems

Item	Requirement
Oracle version	11.2+ for core queries (<code>v\$diag_alert_ext</code> , <code>v\$listener_network</code>) [Likely]; 12c+ for storage_provider (FETCH FIRST) [Certain]
Host OS	Linux (df, ps, free, vmstat, /proc used over SSH)
ASM	Optional; requires opc user with sudo to grid
Data Guard	Optional; standby gap read via SSH on the primary host
Growth chart	Table DASHBOARD_GROWTH_TRACK — not created by this repo; without it the chart shows fixed fallback values

13.3 Network ports (outbound from the client)

Destination	Port	Protocol	Purpose
Primary, reporting, standby DB	Listener port (default 1521)	Oracle Net	SQL queries
Database hosts	22	SSH	OS metrics, ASM, standby gap, RMAN clean-up
SMTP server	465 or 587	SMTPS / STARTTLS	Alert email
localhost	8501-8519	HTTP	Browser to Streamlit

13.4 Database privileges

Current behaviour: registry username sys connects AS SYSDBA. Recommended least-privilege account for read-only monitoring (kill session, RMAN, scheduler and Java features disabled or run separately):

```
CREATE USER dash_mon IDENTIFIED BY "<strong password>";
GRANT CREATE SESSION TO dash_mon;
GRANT SELECT_CATALOG_ROLE TO dash_mon;           -- V$ and DBA_views [Likely
sufficient]
GRANT SELECT ON SYS.V_$DIAG_ALERT_EXT TO dash_mon; -- if not covered by the
role
```

Feature	Extra privilege
Kill session	ALTER SYSTEM
Archive Log Clear (RMAN)	OS authentication, SYSDBA or SYSBACKUP plus an SSH OS account
dashboard_pkg (definer rights)	CREATE PROCEDURE and direct SELECT grants on each SYS.V_\$ view
Scheduler mount path (unused)	CREATE JOB, CREATE EXTERNAL JOB, CREATE ANY DIRECTORY, EXECUTE on UTL_FILE
Java bridge (unused)	CREATE PROCEDURE, JVM, JAVA_ADMIN

14. Function reference

This reference lists every non-trivial function by module with its line number, what it returns and what it touches; functions marked Unused have no caller in the repository (dist/ excluded).

14.1 run_dashboard.py

Function	Line	Purpose	Returns	Side effects
get_reporting_db_config_standalone(db_name, registry_path)	34	Reporting-DB settings from the registry (helper mode)	dict or None	Reads registry
check_connection(cfg)	109	Thick-mode connect: SYSDBA, default, then SID	status dict	init_oracle_client, TCP ping, connect, debug log
is_port_in_use(port)	347	Is a local port listening	bool	Socket

Function	Line	Purpose	Returns	Side effects
open_browser(port)	353	Opens Firefox or default browser	None	Popen / webbrowser
main	368	Port selection, force refresh, start Streamlit	exit code	Writes force_refresh.flag

14.2 app.py

Function	Line	Purpose	Returns	Side effects
module top level	4–296	CLI mode, monitor start, startup registry check, ? shutdown / ? fresh_session / ? selected_db	—	start_monitor_thread, session state
_deferred_exit()	106	Waits for monitor idle, then exits	—	os._exit(0)
load_css()	300	Injects assets/style.css and dark overrides	None	st.markdown
render_sidebar()	535	Theme, navigation, colour-coded DB buttons, registry history, toasts, reset	None	Parallel status fetch (30 workers), email_notifications.json, registry config
render_mail_history()	1031	Gated mail history page	None	Reads email_history.json / alerts_log.txt; Clear writes []
start_session_monitor_thread()	1304	Starts idle-exit watcher once	None	Daemon thread
monitor_sessions()	1311	Exits after 10 s with no sessions	—	Uses Streamlit private runtime API
main()	1362	Page router	None	Session state

Known issue: line 15 calls `db_connection._check_reporting_db_status_direct`, which does not exist; the real check runs through `run_dashboard.py`.

14.3 db_connection.py

Function	Line	Purpose	Returns	Side effects
get_bundled_oci_key(host='')	39	Finds SSH key in keys/ by host, preferred name or first file	path or ""	Lists keys/
_log_normal_db / _log_reporting_db	83 / 101	UP/DOWN logging	None	normal_db_debug.log (passwords when DOWN)
_log_standby_db	138	Standby status and gap logging	None	normal_db_debug.log (masked)
_ensure_thick_mode()	221	One-time Thick init from first folder with oci.dll	bool	PATH, add_dll_directory, init_oracle_client, lock
get_oracle_mode(username)	333	Maps sys/sysoper/sysbackup/sysdg/syskm to auth mode	oracledb mode	—
get_txt_path()	375	Registry path: session → global → db_path_config.json	path or None	Session state
get_registry_history()	421	Previous registry paths	list	May rewrite config
get_pending_path / save_pending_path / confirm_pending_path / discard_pending_path	455–504	Pending-path workflow (save/confirm Unused)	varies	db_path_config.json
clear_active_registry()	518	Moves active path into history (max 10)	None	Config file
startup_registry_check()	542	Discards leftover pending path	None	Config file
save_txt_path(path)	560	Activates a registry; clears caches	None	Config, force_refresh.flag, session caches
parse_smart_txt_to_df(filepath)	634	Whitespace registry parser with NC groups	DataFrame	Reads file
read_db_file_to_df(path)	688	.txt/.csv/.xlsx/.xls with optional header	DataFrame	Reads file
_read_db_file_to_df_cached(path)	768	Cache keyed by path + mtime	DataFrame	Global cache, lock
load_db_names() / load_db_names_api()	785 / 834	Unique DB names, skipping nc	list	—
get_config_for_db(db_name)	880	Primary credentials	dict or None	—
get_reporting_db_config(db_name)	980	Reporting credentials, accepts alias columns	dict or None	—
get_standby_db_config(db_name)	1071	Standby credentials	dict or None	—
check_reporting_db_status(db_name)	1148	Reporting check in subprocess, 15 s timeout	status dict	scratch/rpt_* files
get_api_connection(db_name)	1285	Direct connect with SID fallback	(conn, err)	Connect, log
load_db_config_for_selected()	1381	Credentials for selected DB	dict or None	—

Function	Line	Purpose	Returns	Side effects
<code>get_connection_pool()</code>	1430	1–5 pool for selected DB	pool or None	Global <code>_pool</code>
<code>SSHCursor / SSHConnection</code>	1467–1506	Cursor-like wrapper running SQL via sqlplus over SSH	—	SSH
<code>_build_standby_script(...)</code>	1673	Remote standby script with shlex-quoted values	str	—
<code>_run_standby_ssh_query(...)</code>	1758	Primary host → <code>.env</code> → sqlplus → standby	(records, cols)	SSH exec, 75 s
<code>_open_ssh_client(cfg)</code>	1787	Key-only SSH: config, bundled, <code>~/.ssh</code> , <code>~/.oci</code>	paramiko client	SSH connect
<code>_execute_query_via_ssh(cfg, query, ...)</code>	1841	SQL via host sqlplus	(records, cols)	Password on command line
<code>get_db_connection()</code>	1952	Pool → direct → SSH fallback	conn or None	—
<code>execute_query / execute_query_to_df</code>	1981 / 2012	SELECT with binds	rows / DataFrame	DB
<code>execute_proc_to_df(proc_name, ...)</code>	2025	Calls <code>SYS_REFCURSOR</code> procedure	DataFrame	callproc
<code>execute_non_query(query, params)</code>	2072	DDL/DML on selected DB	(bool, err)	Changes DB

14.4 monitor_thread.py

Function	Line	Purpose	Returns	Side effects
<code>_run_query(conn, sql)</code>	56	Rows as dicts; never raises	list[dict]	DB
<code>collect_db_metrics(db_name, host_cache=None)</code>	73	All metrics for one DB	stats dict	Connect, TCP, reporting subprocess, standby SSH, log-gap email
<code>_write_status(...)</code>	497	Monitor state	None	<code>daemon_status.json</code>
<code>read_monitor_status()</code>	510	State for UI	dict	—
<code>read_monitor_cache()</code>	521	Unused	dict	—
<code>_monitor_loop()</code>	535	120 s cycle, 8 workers, cache + alerts + report	—	Cache file, alerts, report

Function	Line	Purpose	Returns	Side effects
start_monitor_thread / stop_monitor_thread	641 / 657	Lifecycle	None	Thread, _stop_event
is_monitor_idle()	668	No DB in progress	bool	—
trigger_force_refresh()	673	Wake loop	None	force_refresh.flag

14.5 generate_db_status_log.py and monitor_daemon.py

Function	Line	Purpose
tcp_ping(host, port)	21	2 s TCP probe
check_db(cfg, label)	29	Oracle connect, then SSH reachability test
get_standby_sync_status(primary_cfg, standby_cfg)	117	Primary connect + standby gap via SSH
generate_report()	202	Overwrites db_status_report.log
monitor_daemon.collect_db_metrics / main_loop	34 / 438	Legacy sequential 60 s monitor (not used by the app)

14.6 dashboard/home.py

Function	Line	Purpose	Notes
open_db_dialog(db_name)	15	Confirm dialog	Unused
diagnose_registry_connections()	32	Column check + TCP test	Unused (diag_result hard- coded to ok)
load_db_status_summary(db_name)	117	Full status incl. alerts	Unused
seed_session_cache_from_file()	336	Empty caches (disk deliberately ignored)	—
check_standby_db_status_live(db_name)	356	Standby sync	DB + SSH
load_db_status_summary_basic(db_name)	393	Card data	Live SQL, TCP
fetch_host_top_processes(host, ssh_meta)	570	Host top processes	SSH or DB
load_server_status_summary(host, dbs, cache)	615	Host resources, processes, ASM, mounts	Several SSH sessions
determine_health_category(stats)	792	Healthy/Warning/Critical/Unknown	—

Function	Line	Purpose	Notes
build_db_card_html / build_db_card_loading_html	810 / 1087	Card HTML	Not HTML-escaped
render_home_dashboard_fragment(...)	1105	Staged fetch + KPIs + cards (30 s fragment)	Pools 30/5/5/5
render_server_details_for_host(...)	1441	Host panels	Nested
render_home_page()	1807	Setup screen or portal	Uploads to scratch/<filename> unsanitised

14.7 dashboard/monitoring.py

Function	Line	Purpose	Notes
get_drive_details(conn=None)	32	Volumes + ASM	SSH df
update_session_history(current, hwm)	105	24 h session history	Seeds new file with random values
render_status_card / render_session_card	159 / 181	Status widgets	—
render_tablespace_bars(df_ts, cols_count=4)	206	Up to 12 bars with copyable location	—
clear_archive_logs_via_ssh(db_name)	315	RMAN delete archivelogs > 15 days	Destructive; unescaped shell values
get_standby_gap_live(db_name, do_log=True)	447	Live standby gap	DB + SSH
_render_top_processes_fragment(db_name)	528	Top CPU/memory tables, 30 s	SSH or DB
render_dashboard(db_name)	650	Detail page, 18 parallel fetches, kill session, archive clear	ALTER SYSTEM KILL SESSION, SWITCH LOGFILE

14.8 dashboard/mail_config.py and utils/alerts.py

Function	Line	Purpose
mail_config._load_prefs / _save_prefs	115 / 121	Read/write preferences in config/<registry>.json
mail_config.render_mail_config_page()	132	SMTP form, save, test email, preferences
alerts.get_custom_config_path()	30	config/<registry>.json path
alerts.is_alert_enabled_by_user(key)	58	Preference check, default False
alerts.load_config / load_state / save_state	76 / 98 / 107	Config and alert_state.json
alerts._write_notification	118	Toast queue (max 20)
alerts._write_history	144	History (max 200)
alerts.send_alert_email(subject, body, alert_type=None)	199	Preference check, log, SMTP send; returns {success, message, error}
alerts.check_and_trigger_alerts(server, db, host, stats)	343	State-change evaluation of the 7 implemented alert types

14.9 utils providers

Function	Module:line	Purpose
get_asm_diskgroup_info(host, password, key_filename, port)	asm_provider:521	60 s cached ASM disk groups
_fetch_asm_diskgroups_via_ssh(...)	asm_provider:232	opc → sudo su - grid → sqlplus as sysasm
get_mounts_via_ssh(host, username, ...)	ssh_mount_provider:166	df -PkT + /proc/mounts
get_mounts_via_oracle_scheduler(conn)	ssh_mount_provider:308	Unused; creates DASH_TMP_DIR and DASH_DF_JOB

Function	Module:line	Purpose
get_top_processes_for_db / _for_host	ssh_process_provider:612 / 746	Top processes via SSH with DB fallback
get_top_processes_via_db_only(db_name)	ssh_process_provider:305	Two v\$sesstat samples 1 s apart
get_system_resources_via_ssh(host, username, ...)	ssh_resource_provider:10	CPUs, RAM, CPU %
get_storage_provider(conn)	storage_provider:821	Picks provider by platform
StorageProvider.get_storage_info()	storage_provider:31	Mounts over SSH using registry host/key
StorageProvider.deploy_java_bridge()	storage_provider:261	Unused; Java DDL and file grants
StorageProvider.get_process_info()	storage_provider:494	Unused; SSH branch broken by bad import

15. Query and PL/SQL catalogue

queries/queries.py holds 27 SQL constants, all of them read-only; dashboard_pkg.sql mirrors most of them as 26 procedures but is effectively unused at runtime.

15.1 queries.py

Constant (line)	Views	Output columns	Filter / limit	Used by
_SQL_DB_STATUS (11)	v\$instance	status, instance_name, host_name, version, startup_time	—	home, monitoring, app
_SQL_SESSION_STATS (23)	v\$session	status, cnt	username not null, type='USER'	home, monitoring
_SQL_TABLESPACES (32)	dba_data_files, dba_free_space	tablespace_name, allocated_mb, free_mb, used_mb, maxsize_mb, free_on_max_mb, pct_used_max, location	—	home, monitoring

Constant (line)	Views	Output columns	Filter / limit	Used by
_SQL_BACKUP_STATUS (77)	v\$rman_backup_job_details	status	latest row	monitoring
_SQL_RMAN_DURATIONS (86)	v\$rman_backup_job_details	start_time_str, duration_min, status, input_type	last 7 days	home, monitoring
_SQL_SESSION_LICENSE (97)	v\$license	sessions_current, sessions_highwater	—	monitoring
_SQL_DB_GROWTH (102)	DASHBOARD_GROWTH_T_RACK	record_date_str, used_mb, total_mb	—	monitoring
_SQL_CPU_SESSIONS (115)	v\$session, v\$sesstat, v\$statname, v\$sql	sid, username, program, cpu_time_val, sql_text, sql_id	USER, not KILLED, system users excluded, top 10	monitoring
_SQL_BLOCKING_SESSIONS (138)	v\$session ×2, v\$sql	blocking/blocked sid, user, program, wait info, blocked SQL	blocked only	home, monitoring
_SQL_LOCK_WAITS (159)	v\$session, v\$sql, v\$lock	sid, user, wait, lock_type, request, mode, SQL	blocking or Application/Concurrency waits	monitoring
_SQL_ALERT_LOG (182)	v\$diag_alert_ext	time_str, message_text	last 48 h, ORA-, ≤ 50	monitoring
_SQL_ARC_LOG (194)	v\$archived_log, v\$parameter, v\$recovery_file_dest	archive_logs, size_gb, FRA limit/used	deleted='NO'	Unused
_SQL_MAX_SESSIONS (205)	v\$parameter	name, value	sessions	home, monitoring
_SQL_ASM_STORAGE (210)	v\$asm_diskgroup, v\$asm_disk	disk_group, total_mb, free_mb, state, type, disk_count	—	monitoring
_SQL_OS_STATS (221)	v\$osstat	stat_name, value	5 stats	home, monitoring
_SQL_SYSMETRIC (227)	v\$sysmetric	metric_name, value	3 metrics, group 2	home, monitoring
_SQL_ORACLE_MEM (234)	v\$sga, v\$process	sga_bytes, pga_bytes	—	home, monitoring
_SQL_SGA_TOTAL / _SGA_FREE / _PGA_STATS (242-248)	v\$sga, v\$sgastat, v\$pgastat	totals, free, PGA stats	—	monitoring
_SQL_LISTENER_NETWORK / _INSTANCE (255-256)	v\$listener_network, v\$instance	type, value / status	first row	home, monitoring
_SQL_ARC_DEST / _FRA / _CUSTOM (259-267)	v\$archive_dest, v\$recovery_file_dest, v\$archived_log	destination, FRA GB and %, log count/size	dest 1	home, monitoring
_SQL_PID_SID_MAP (270)	v\$process, v\$session	spid, sid, serial#, username, program	—	Unused
_SQL_PRIMARY_LOG_SEQ (279)	v\$archived_log	thread#, primary_last_generated	per thread	monitor, monitoring, report

Constant (line)	Views	Output columns	Filter / limit	Used by
_SQL_STANDBY_LOG_GA P (290)	v\$archived_log on standby via SSH	thread#, received, applied, gap	dest 1	monitor, monitoring, report
_SQL_ARCHIVE_DEST_ST ATUS (306)	v\$archive_dest_status	dest_id, name, status, error, db_unique_name, sync status	dest 1, 2	monitor, monitoring, report

Query notes:

- get_db_growth_rates returns fixed values (2.9, 20.3, 90.7, 1088.3 MB) when DASHBOARD_GROWTH_TRACK has fewer than 2 rows; nothing in the repo creates or fills that table.
- Blocking and lock-wait queries still left-join v\$sql by sql_id and can return duplicate rows per child cursor.

15.2 Inline SQL

Location	Purpose	Views
monitor_thread.py:126	Listener	v\$listener_network
monitor_thread.py:133	Latest backup and age	v\$rman_backup_job_details
monitor_thread.py:162	Tablespaces	dba_data_files, dba_free_space
monitor_thread.py:204– 219	Archive destination and FRA	v\$archive_dest , v\$recovery_file_dest , v\$archived_log
monitor_thread.py:232	Blocking (≤ 10)	v\$session
monitor_thread.py:243 / 254	Deadlock (last hour, ≤ 5) / row-lock fallback	v\$diag_alert_ext / v\$session_wait
monitor_thread.py:263	ORA errors (48 h, ≤ 50)	v\$diag_alert_ext
monitor_thread.py:277– 279	CPU, RAM, SGA+PGA	v\$osstat , v\$sysmetric , v\$sga , v\$process
monitor_thread.py:317 / 328 / 347	Sessions limit / standby dest errors / log sequences	v\$parameter / v\$archive_dest_status / v\$archived_log
home.py:441 / 453 / 464 / 494	Instance, active sessions, latest RMAN job, tablespaces	v\$instance , v\$session , v\$rman_backup_job_details , dba_data_files

15.3 dashboard_pkg (queries/dashboard_pkg.sql)

Every procedure takes one parameter, p_recordset OUT SYS_REFCURSOR. The only caller is monitor_daemon.py:215 (get_oracle_mem), which is broken and not on the app's path.

Group	Procedures	Difference from queries.py
Status and sessions	get_db_status, get_session_stats, get_session_license, get_max_sessions, get_cpu_sessions, get_blocking_sessions, get_lock_waits, get_pid_sid_map	get_session_stats and get_cpu_sessions lack type='USER'; get_cpu_sessions keeps the duplicate-row join; get_pid_sid_map returns os_pid, oracle_sid, username
Storage	get_tablespaces, get_asm_storage, get_arc_log, get_arc_dest, get_arc_fra, get_arc_custom, get_db_growth	Same as Python
Backup	get_backup_status, get_rman_durations	Same
Host and memory	get_os_stats, get_sysmetric, get_oracle_mem, get_sga_total, get_sga_free, get_pga_stats	Same
Listener	get_listener_network, get_listener_instance	Same
Alert log	get_alert_log	Same
Data Guard	none	Not in package

Installation (only if the package is adopted): compile as the monitoring owner with CREATE PROCEDURE and direct SELECT grants on each `SYS.V_$` view, because definer-rights code ignores roles.

16. Alert type catalogue

The Email Configuration page offers 32 alert checkboxes, but only 7 are evaluated by the alert engine; one more is wired but can never fire, and the Data Guard log-gap alert has no checkbox at all.

16.1 Sending rules (all types)

- Alerts are sent only by the background monitor (monitor_thread.py:608), every 120 s.

- `is_alert_enabled_by_user(key)` defaults to `False`, so nothing is sent until preferences are saved.
- An unchecked type returns before the audit log and history are written; the UI tip "events are still logged" (`mail_config.py:342`) is inaccurate.
- De-duplication state in `config/alert_state.json` is keyed `host:db_name`. The flag is set even when sending was skipped or failed, so enabling a type while a condition persists sends nothing until it clears and recurs.

16.2 Implemented alert types

Key	Trigger	Threshold	Re-arm	Subject
<code>db_down</code>	Connection fails (incl. bad credentials)	—	When DB not DOWN	Critical Alert - Database Down
<code>listener_down</code>	No row from <code>v\$listener_network</code> , or TCP to port fails	—	When UP	Critical Alert - Listener Down
<code>tablespace_critical</code>	<code>pct_used_max</code> of a permanent tablespace	$\geq 90\%$	Per tablespace, below 90%	Critical Alert - Tablespace Nearly Full
<code>archive_log_full</code>	FRA used / limit (FRA destination only), or archiver error text	$\geq 90\%$	When clear	Critical Alert - Archive Log Issue
<code>blocking_sessions</code>	Any blocked session	≥ 1	When none	Critical Alert - Blocking Session Detected
<code>deadlock</code>	Alert log line containing deadlock in last hour	≥ 1	When none in last hour	Critical Alert - Deadlock Detected
<code>ora_error</code>	ORA- lines in alert log, last 48 h, first 10	Any	Never; last 50 codes remembered	Critical Alert - ORA Error Detected

16.3 Partly implemented

Key	Status
mount_critical	Logic exists ($\geq 90\%$, pseudo filesystems skipped) but stats["drives"] is always empty in the monitor, so it never fires
log_gap_alert (no checkbox)	Sent when max Data Guard gap ≥ 2 sequences; always suppressed because the key cannot be enabled from the UI; no de-duplication if enabled by hand; monitoring.py:1813 still says "Email alert triggered"

16.4 UI-only alert types (never evaluated)

Group	Keys
Database Availability	db_not_reachable, db_restarted, host_down
Backup	backup_failure (FAILED is computed but not mailed)
Tablespace & Datafiles	tablespace_warning, undo_tablespace_critical, temp_tablespace_critical, datafile_autoextend
Archive Log & FRA	archive_log_failure, fra_critical (both covered by archive_log_full)
Sessions & Locks	session_maxed (computed but not mailed), long_running_sessions
CPU & Memory	high_cpu, high_memory, cpu_process_threshold, mem_process_threshold
Storage & Mount Points	mount_warning, filesystem_not_accessible, disk_critical, asm_critical
Network & Connectivity	listener_not_reachable, ssh_failure
Errors & Objects	alert_log_errors, invalid_objects

16.5 Email format

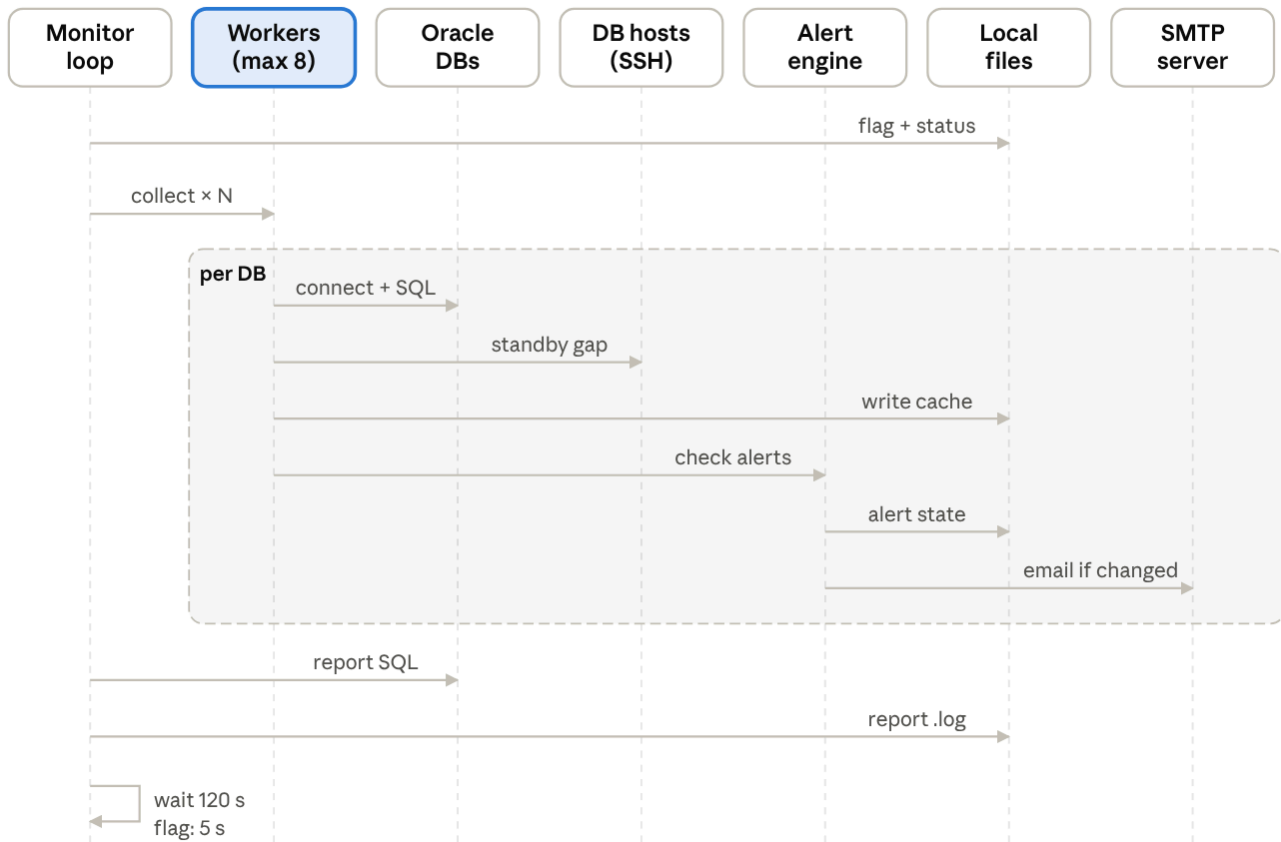
Plain text (EmailMessage.set_content), no HTML.

Style	Used by	Content
A	db_down, listener_down, tablespace, archive, mount	Server Name, Database Name, Host Name, Alert Type, Severity: Critical, Status, Date & Time, then detail lines (tablespace: name, used/free/max MB, %; mount: mount point, total/used/free GB, %; others: Problem)
B	ora_error, deadlock, blocking	Padded labels, no Status line, details (ORA code and message, or up to 5 SID/Serial/User/Blocking lines), ending with Action
Data Guard	log_gap_alert	Database, Standby DB, Maximum Log Gap, per-thread generated/received/applied/gap, Action Required, Time
Test	Send Test Email	Subject "  Test Alert — DB Health Dashboard"; SMTP server:port, From, To, Sent At; bypasses preferences

17. Monitoring cycle sequence

Each cycle runs every 120 seconds: up to 8 databases are checked in parallel, each result is cached and passed to the alert engine, and a full status report is written at the end.

Each 120-second cycle checks every database, then alerts only on change



One monitoring cycle · `monitor_thread._monitor_loop`

Step	Actor	Action	Code
1	Monitor loop	Delete <code>force_refresh.flag</code> ; write <code>cycle_start</code> to <code>daemon_status.json</code>	<code>monitor_thread.py:545-548</code>
2	Monitor loop	Submit <code>collect_db_metrics</code> for every database to <code>ThreadPoolExecutor(8)</code>	<code>:581</code>
3	Worker	Connect (5 s TCP timeout), run metric SQL; SSH fallback if needed	<code>:73-493</code>
4	Worker	Standby gap over SSH via the primary host; reporting check in a subprocess	<code>:445;</code> <code>db_connection.py:1148</code>
5	Worker	Rewrite <code>db_monitor_cache.json</code> under <code>_cache_write_lock</code>	<code>:595-600</code>
6	Worker	<code>check_and_trigger_alerts</code> under <code>_alert_lock</code>	<code>:606-608</code>
7-8	Alert engine	Update <code>alert_state.json</code> ; email only on state change	<code>utils/alerts.py:343</code>

Step	Actor	Action	Code
9–10	Monitor loop	generate_report() reconnects to every DB and overwrites db_status_report.log	:621–622
11	Monitor loop	Wait 120 s, checking for force_refresh.flag every 5 s	:630–635

18. Session state and caching reference

All UI caching lives in `st.session_state`, so it is per browser session and lost on reload; only the monitor's results persist on disk.

18.1 Session state keys

Area	Keys	Purpose
Navigation	page, selected_db, selected_chk_db, active_detail_db, dialog_open	Current screen and database
Theme	theme	Light or Dark
Setup	_startup_registry_checked, session_config_done, monitoring_started, force_config_screen, custom_txt_path	Registry load and setup flow
Shutdown	app_shutting_down	Shows the shutdown overlay
Mail History	mail_history_authenticated	Gate passed for this session
Home caches	status_cache, server_cache, home_load_complete, home_processes_cache, home_mounts_cache, diag_result	Card, host and diagnostics data
Detail cache	mon_cache_<db>	Monitoring page data, reused for 60 s
Refresh	refresh_mode, refresh_interval, last_refresh_time, _refresh_cycle_start, force_live_refresh	Manual/automatic refresh control
Monitoring actions	killed_sids, kill_sid_input, cpu_sel_*_<db>, standby_logged<db>	Kill session and selection state

Area	Keys	Purpose
Mail config	hf_multisel_widget, mc_*, mc_pref_<key>	Form fields and preference checkboxes
Alerts	alert_*	Cleared on new session or registry reset

18.2 Cache layers

Layer	Scope	Lifetime	Invalidated by
Registry parse cache	Process memory	Until file modification time changes	Editing the registry file
status_cache / server_cache	Browser session	Until Refresh or Portal Home	Refresh button, registry change
mon_cache_<db>	Browser session	60 s	Age, Refresh
Process / mount caches	Process memory	30 s	Age
ASM cache	Process memory	60 s	Age
db_monitor_cache.json	Disk	Rewritten every cycle (120 s)	Refresh deletes it and forces a cycle

18.3 Refresh timers

Timer	Interval	Mechanism
Home card fragment	30 s	@st.fragment(run_every=30)
Top processes fragment	30 s	@st.fragment in monitoring.py
Automatic page refresh	User-set, default 20 min, minimum 1	Hidden JS countdown clicks Refresh
Background monitor	120 s	monitor_thread.py loop, 5 s flag checks
Idle shutdown	10 s with no sessions	app.py session watcher

19. Testing and verification

The project has no automated test suite; test_oci.py (24 lines, hard-coded lib_dir) only checks that the Instant Client loads, so every release must be verified manually with the cases below.

19.1 Test environment

- One Windows 10/11 PC without Python or Oracle installed, for the packaged exe.
- At least one primary database with a reporting copy and a physical standby, reachable on 1521.
- SSH access to each database host; an SMTP relay or test mailbox.
- A registry file covering: all three groups filled; reporting = NC; production = NC; one deliberately wrong host password.

19.2 Release test cases

ID	Area	Steps	Expected result
TC-01	Install	Unzip release, run DB_Health_Portal.exe	Console opens; browser opens on localhost:8501 within ~5 s
TC-02	Port reuse	Run the exe a second time	Second run opens a tab to the running instance and exits
TC-03	Port fallback	Occupy 8501 with another service, run exe	App starts on first free port 8502–8519

ID	Area	Steps	Expected result
TC-04	Registry upload	Upload .txt, .csv and .xlsx variants	Databases listed; NC groups show NONE; NC production omitted
TC-05	Registry path	Enter an absolute path, Connect Path	Same as TC-04; path saved in db_path_config.json history
TC-06	Last used	Restart, press Connect & Use Last	Loads without re-entering the path
TC-07	DB down	Stop a listener or block port 1521	Card turns red within one cycle (≤ 2 min); DB-down email once
TC-08	Recovery	Restore the database	Card returns to green; alert re-arms
TC-09	Tablespace	Fill a test tablespace above 90%	Card amber with tablespace name; email once
TC-10	Blocking	Create a row-lock between two sessions	Warning status; red bar in Top 10 CPU Sessions; email once
TC-11	Standby gap	Defer log shipping until gap ≥ 2	Standby NOT-SYNC; card red
TC-12	SSH failure	Wrong host password in registry	Card error "host username and password is wrong"; DB metrics still shown
TC-13	Kill session	Kill a test session from the UI	Session ends; SID listed as killed
TC-14	Kill input validation	Enter a non-numeric SID	Must be rejected (currently not — see 9.2 #6)
TC-15	Archive clear	Run Archive Log Clear on a test DB	Only logs older than 15 days deleted; result shown
TC-16	Email test	Configure SMTP on port 465 and 587, Send Test Email	Test mail arrives using the configured account on both ports (587 currently uses the hard-coded account — 9.2 #1)
TC-17	Preferences	Disable an alert type, trigger it	No email and no Mail History entry
TC-18	Mail History gate	Wrong then correct sender/password	Rejected, then unlocked

ID	Area	Steps	Expected result
TC-19	Refresh	Manual Refresh; Automatic at 1 min	Data reloads; countdown triggers reload
TC-20	Shutdown	Shutdown button; close all tabs	Process exits; after tab close within ~10 s
TC-21	Secrets	Search logs, cache and scratch/ for test passwords	No plaintext passwords (currently fails — 9.2 #4, #5)

19.3 Recommended automated tests

- Unit: parse_smart_txt_to_df with NC in each group, short and long lines, csv/xlsx headers.
- Unit: alert state machine — send once, re-arm on clear, ORA code memory.
- Unit: health classification rules in home.py.
- Integration: Oracle Free container with a seeded blocking session and a full tablespace.

20. Appendices

Appendix A — Sample registry lines

All values below are placeholders. Fields are whitespace-separated; values must not contain spaces.

```
# All three groups configured
PRODDB 10.0.0.10 1521 PRODDB_SVC sys <db_pwd> oracle <host_pwd> RPTDB
10.0.0.20 1521 RPTDB_SVC sys <db_pwd> STBYDB 10.0.0.30 1521 STBYDB_SVC sys
<db_pwd>

# Reporting not configured (one NC token)
PRODDB 10.0.0.10 1521 PRODDB_SVC sys <db_pwd> oracle <host_pwd> NC STBYDB
10.0.0.30 1521 STBYDB_SVC sys <db_pwd>

# Reporting and standby not configured
PRODDB 10.0.0.10 1521 PRODDB_SVC sys <db_pwd> oracle <host_pwd> NC NC
```

Parsing rules: blank lines and lines starting with # are skipped; .txt has no header detection; .csv/.xlsx treat the first row as a header only when its third field (port) is non-numeric; port defaults to 1521 for primary and standby; service_name defaults to db_name for primary.

Appendix B — Configuration file schemas

File	Type	Keys
db_path_config.json	object	txt_path, history (list, max 10), pending_txt_path
config/<registry>.json, email_config.json	object	smtp_server, smtp_port, smtp_username, smtp_password, from_email, to_emails, enabled, preferences{key: bool}
config/db_monitor_cache.json	object keyed by db_name	db, listener, backup, full_ts, balance_ts, arc_* (configured, dest_name, limit/used/free in MB and GB, pct, log_type), has_blocking, blocking_details, deadlock_detected, alert_log_errors, ora_errors, system_res, processes, drives, session_maxed, standby_configured, standby_dests, standby_log_gap, primary_seqs, log_gap_alert, reporting_* (configured, db_name, status, listener, username, password, error_message, response_time, checked_at), tooltip_reasons, last_refresh
config/alert_state.json	object keyed host:db_name	db_down, listener_down, tablespaces, arc_log_alert, mount_points, ora_errors_sent, deadlock_alert, blocking_alert
config/daemon_status.json	object	fetching, last_cycle_start, last_cycle_end
config/email_history.json	array (max 200)	timestamp, subject, alert_type, db_name, server, to, status, error, body_preview
config/email_notifications.json	array (max 20)	timestamp, type, subject, message, seen

File	Type	Keys
.env	dotenv	ORACLE_USER, ORACLE_PASS, ORACLE_DSN (not read by current code)

Stale files: config/db_name.json, config/server_names.json and "config/kasorcl 10.10.10.11 1521 kasorcl sy.json" are mail configs created from registry files with unusual names; review and delete them.

Appendix C — Glossary

Term	Meaning
ADR	Automatic Diagnostic Repository; source of v\$diag_alert_ext
ASM	Automatic Storage Management
Easy Connect	host:port/service_name connect string
FRA	Fast Recovery Area
NC	Not Configured marker in the registry
NNE	Oracle Native Network Encryption (reason Thick mode is bundled)
Registry file	Text/CSV/Excel list of monitored databases
Thick / Thin mode	python-oracledb with or without Oracle Client libraries
SID / serial#	Oracle session identifiers used to kill a session
SYSDBA	Oracle administrative privilege